

ReAct: 让大模型同时"思考"和"行动"

论文: ReAct: Synergizing Reasoning and Acting in Language Models

作者: Shunyu Yao (Princeton) 等, Google Brain

发表: ICLR 2023 (顶会)

arXiv: 2210.03629v3

一句话总结: 让大语言模型在解决任务时, 把"推理" (想一想) 和"行动" (做一做) 交替进行, 两者互相促进, 比单独推理或单独行动都强。

一、这篇论文解决了什么问题?

在 ReAct 之前, LLM 的两大能力是分开研究的:

能力	代表方法	问题
推理 (Reasoning)	Chain-of-Thought (CoT, 思维链)	只在脑子里想, 容易"幻觉"——编造不存在的事实
行动 (Acting)	WebGPT, SayCan	只会做, 不会反思, 容易迷失方向或重复犯错

ReAct 的核心洞察: 人类做事时, 思考和行动是交替进行的。比如做饭——切完菜后你会想"下一步该烧水了" (推理), 然后去开火 (行动), 看到水开了 (观察), 再想"该下面条了" (推理) ……这个循环才是智能的关键。

二、核心方法 (只需要理解这一页)

2.1 核心思想: 扩展行动空间

原来的 Agent (智能体): 观察 → 行动 → 观察 → 行动...

ReAct 的 Agent: 观察 → **思考** → 行动 → 观察 → **思考** → 行动...

用数学语言说: 把行动空间从 A 扩展为 $\hat{A} = A \cup L$, 其中 L 是自然语言空间。"思考" (Thought) 是一种 **不影响外部环境** 的语言行动, 只用来更新自己的内部状态。

2.2 直觉理解

想象你是工厂里的机械臂操作员, 收到一个模糊指令: "把那个零件放过去"。

- 只推理 (CoT):** 你在脑子里猜"那个零件"是什么, "那里"在哪——但可能猜错
- 只行动 (Act-only):** 你直接乱抓, 碰运气——效率极低
- ReAct:** 你先想"那个零件可能是螺丝或螺母" (Thought), 然后去看一眼桌面 (Action→Observation), 看到只有螺丝, 于是确认"就是这个螺丝" (Thought), 再执行抓取 (Action)

2.3 实现方式：Few-shot Prompting（少样本提示）

不需要训练！只需给 LLM 看几个人写的"思考-行动-观察"交替的例子，LLM 就学会了这种模式。

```
Thought 1: 我需要搜索 Apple Remote，找到它最初设计用来控制什么程序。
Action 1: Search[Apple Remote]
Obs 1:  Apple Remote 是一款遥控器……最初设计用来控制 Front Row 媒体中心程序……
Thought 2: Apple Remote 最初设计用来控制 Front Row。我需要搜索 Front Row，找出还有什么设备能控制它。
Action 2: Search[Front Row]
...
```

三、关键实验结果（记住结论就行）

3.1 知识推理任务（HotpotQA & FEVER）

方法	HotpotQA	FEVER
CoT（只推理）	29.4	56.3
Act（只行动）	25.7	58.9
ReAct	27.4	60.9
ReAct + CoT-SC	35.1	64.6

关键发现：ReAct 和 CoT 各有所长，**组合使用效果最好**——当 ReAct 搜索失败时回退到 CoT（用内部知识），当 CoT 不够自信时回退到 ReAct（去外部查）。

3.2 错误分析（必须理解）

失败类型	ReAct	CoT
推理错误	47%	16%
幻觉（编造事实）	0%	56%
搜索失败	23%	—

核心对比：

- CoT 的最大问题是**幻觉**（超过一半的错误），ReAct 因为能查外部知识，幻觉为 0
- ReAct 的最大问题是**推理错误**（容易陷入循环），因为交替格式限制了推理的灵活性

3.3 决策任务（ALFWorld & WebShop）

- ALFWorld（文本版家庭任务）：ReAct 71% vs Act-only 45% vs 模仿学习 37%
- WebShop（网上购物）：ReAct 成功率 40% vs 之前最好的 29%

仅用 1-2 个示例提示，就超过了用上万条数据训练的模仿学习/强化学习方法。

3.4 微调结果

用 3000 条 ReAct 自己生成的正确轨迹微调小模型，8B 模型微调后超过 62B 模型的提示效果——说明 ReAct 范式在微调场景下潜力巨大。

四、与你课题的直接联系

4.1 "模糊指令消歧" ↔ ReAct

你的课题核心场景：工人对机械臂说"把那个拧到那里"。

ReAct 给你的启发是：

1. **先想再做**：机械臂收到模糊指令后，不要直接执行，先用 Thought 推理"那个"可能指什么
2. **做了再想**：用 Action 调用视觉模块（如 Grounding DINO）看一眼现场，得到 Observation，再用 Thought 缩小候选范围
3. **失败了就问**：如果 Thought 判断候选太多无法确定，触发 Action = "向用户澄清"

4.2 在你的技术栈中的位置

用户说"把那个螺丝拧到那里"

|



[ReAct 式推理-行动循环]

|

Thought: "那个"可能指螺丝，需要先找候选

Action: 调用 Grounding DINO → 返回 3 个螺丝候选

Obs: 螺丝A(左上), 螺丝B(中间), 螺丝C(右下)

Thought: 3 个候选置信度接近，需要澄清

Action: Ask("您指的是左上、中间还是右下的螺丝? ")

Obs: 用户回答"左上那个"

Thought: 确认目标为螺丝A，执行拧入

Action: 执行抓取和拧入动作

4.3 与已学论文的关系

已学论文	与 ReAct 的关系
SayCan	SayCan 用 affordance（可供性）做 world grounding，ReAct 用外部交互做 knowledge grounding。两者互补——ReAct 提供推理框架，SayCan 提供物理约束
Inner Monologue	ReAct 论文直接对比了 Inner Monologue，指出 IM 的"内心独白"只是在复述环境状态，缺乏真正的高层推理（如目标分解、常识推理）。ReAct 的 Thought 更灵活
VIMA	VIMA 处理多模态指令，ReAct 提供了将推理嵌入行动循环的范式。你可以把 VIMA 的多模态感知作为 ReAct 循环中的 Observation 来源

五、哲学启示

ReAct 体现了一个深刻的认知科学洞察：**知行合一**。

王阳明说"知是行之始，行是知之成"——认知和行动本来就不该割裂。纯粹的思考（CoT）脱离了现实容易幻想；纯粹的行动（Act-only）缺少反思容易盲目。ReAct 把两者交织在一起，正是"知行合一"的计算机实现。

这也暗示了一个 AI 研究的重要趋势：**最好的智能不是最强的大脑，而是最好地与世界互动的能力**。

六、你需要跳过的部分

- Appendix B-E 的所有细节（各种 prompt 模板和失败案例的细节分析）
- GPT-3 的对比实验（Appendix A.1，结论和主实验一致）
- WebShop 的具体 prompt 设计（Table 6, 10）

七、你必须看懂的图

图	位置	为什么重要
Figure 1	第 2 页	整篇论文的灵魂图！4 种方法的对比 + 推理-行动交替的完整例子
Figure 3	第 7 页	微调 vs 提示的 scaling 曲线，说明 ReAct 微调潜力大
Table 2	第 6 页	错误分析对比表，理解 ReAct 和 CoT 各自的软肋

八、术语速查表

英文	中文	一句话解释
ReAct	推理+行动	Reasoning + Acting 的缩写，交替推理和行动的范式
Chain-of-Thought (CoT)	思维链	让 LLM "一步步想"的提示技术
Self-Consistency (SC)	自一致性	多次采样取多数投票，提升 CoT 的鲁棒性
Few-shot Prompting	少样本提示	给 LLM 看几个例子就能学会新任务
Hallucination	幻觉	LLM 编造不存在的事实
Action Space	行动空间	Agent 所有可能采取的行动的集合
Trajectory	轨迹	Agent 的行动-观察序列的完整记录
Bootstrapping	自举	用模型自身生成的数据来训练自己
Grounded	接地的/有据的	基于真实信息而非臆想的
Inner Monologue	内心独白	用环境反馈文字来引导 Agent 的方法